

# Scala

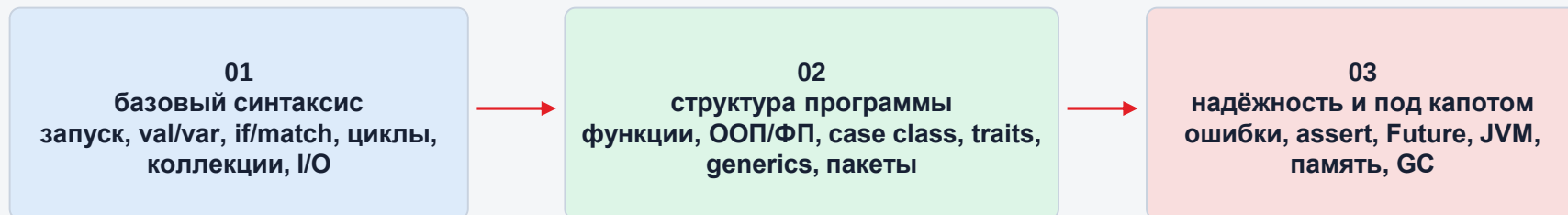
ошибки, конкурентность, JVM и память

Третья презентация: как писать более надёжный код и что происходит под капотом

Команда 4. Бурачевский,  
Киселева, Ятманова.  
Группа 5030102/30202

# Где мы находимся в ТЗ

Первые две презентации закрыли синтаксис и стиль программирования; теперь — надёжность и runtime



## Что осталось изучить

- обработка ошибок
- исключения и assert
- конкурентность и параллелизм
- JVM подробнее
- управление памятью и GC

# Надёжная Scala-программа

---

В третьей части связываем ошибки, проверки, ресурсы, Future и JVM в одну практическую картину

## 1. Ошибки

Как программа сообщает о проблемах: exceptions, Try, Either.

## 2. Проверки

assert, require и простые контракты для входных данных.

## 3. Ресурсы

Файлы и закрытие ресурсов через Using, чтобы не забыть close().

## 4. Конкурентность

Future, ExecutionContext, объединение задач и общая изменяемость.

## 5. JVM и память

Байткод, стек, heap, ссылки, сборщик мусора.

**главная идея: Scala не отменяет runtime-проблемы, но даёт удобные средства для их контроля**

# Как в Scala представлять ошибки

Есть несколько уровней: от “сломалась программа” до контролируемого результата

## Exception

Для неожиданных ситуаций или Java API. Можно throw / catch, но поток исполнения прерывается.

## Option[A]

Значение может отсутствовать: Some(value) или None. Хорошо для “не найдено”.

## Try[A]

Результат вычисления, которое может упасть: Success(value) или Failure(error).

## Either[E, A]

Либо ошибка известного типа E, либо успешное значение A. Хорошо для бизнес-ошибок.

```
def findUser(id: Int): Option[String]

def parseInt(s: String): Try[Int]

def validate(score: Int): Either[String, Int]
```

## Практическое правило

Не всё надо делать через exception. Если ошибка ожидаемая и нормальная для бизнес-логики, лучше сделать её частью типа результата.

# Исключения: try / catch / finally

Scala использует JVM-исключения и может ловить ошибки Java-кода

```
def parsePositive(text: String): Int =  
  val value = text.toInt  
  if value < 0 then  
    throw new IllegalArgumentException("negative")  
  value  
  
try  
  println(parsePositive("42"))  
catch  
  case e: NumberFormatException =>  
    println("not a number")  
  case e: IllegalArgumentException =>  
    println(e.getMessage)  
finally  
  println("finished")
```

## Что важно

- catch использует pattern matching по типу исключения
- finally выполняется в конце
- throw прерывает обычный поток выполнения
- исключения удобны для аварийных ситуаций и Java API

## Осторожно

Если исключение — обычная ожидаемая ветка логики, код становится менее явным. Для таких случаев часто удобнее Try или Either.

# Try[A]: ошибка как значение

Try превращает потенциальное исключение в результат Success или Failure

```
import scala.util.{Try, Success, Failure}

def parseScore(text: String): Try[Int] =
  Try(text.toInt).flatMap { n =>
    if n >= 0 then Success(n)
    else Failure(new IllegalArgumentException("negative"))
  }

parseScore("95") match
  case Success(score) => println(s"score = $score")
  case Failure(error) => println(error.getMessage)

val safe = parseScore("bad").getOrElse(0)
```

## Когда удобно

- парсинг строк
- чтение внешних данных
- вызовы кода, который может бросать exception
- быстрый переход от exception-стиля к более безопасному API

## Методы

map — преобразовать успех  
flatMap — связать шаги  
recover — заменить ошибку  
getOrElse — значение по умолчанию

# Either[E, A]: явная бизнес-ошибка

Either позволяет вернуть не просто “упало”, а осмысленное сообщение или тип ошибки

```
def validateScore(text: String): Either[String, Int] =  
  text.toIntOption match  
    case None => Left("not a number")  
    case Some(n) if n < 0 => Left("negative score")  
    case Some(n) if n > 100 => Left("score > 100")  
    case Some(n) => Right(n)
```

```
val result = validateScore("105")
```

```
val message = result match  
  case Right(score) => s"accepted: $score"  
  case Left(error) => s"error: $error"
```

## Option vs Try vs Either

Option: значения может не быть

Try: вычисление может бросить exception

Either: ошибка ожидаемая и мы хотим описать её явно

## Почему это полезно

Сигнатура функции сама говорит, что может произойти ошибка, и заставляет вызывающий код её обработать.

# assert, require, ensuring

Встроенные проверки помогают явно фиксировать предположения программы

```
def average(scores: List[Int]): Double =  
  require(scores.nonEmpty, "scores must not be empty")  
  assert(scores.forall(_ >= 0), "scores must be non-negative")  
  
  val result = scores.sum.toDouble / scores.length  
  
  result.ensuring(_ >= 0)  
  
  println(average(List(80, 90, 100)))
```

## require

Проверяет входные данные функции. Обычно означает: "вызывающий код передал неправильный аргумент".

## assert

Проверяет внутреннее предположение программы. Если оно нарушилось, скорее всего ошибка в логике.

## ensuring

Проверяет результат выражения. Удобно для постусловий: что должно быть верно после вычисления.

# Безопасная работа с ресурсами: Using

Файлы, соединения и потоки нужно закрывать даже при ошибке

```
import scala.io.Source
import scala.util.Using

val lines = Using(Source.fromFile("students.csv")) { source =>
  source.getLines().toList
}

lines match
  case Success(value) => println(value)
  case Failure(error) => println(error.getMessage)
```

## Проблема без Using

Если во время чтения файла возникнет ошибка, можно забыть вызвать `close()`. Тогда ресурс останется открытым.

## Что делает Using

Открывает ресурс, выполняет блок кода и гарантированно закрывает ресурс после завершения.

**Using возвращает Try, поэтому хорошо сочетается с обработкой ошибок**

# Конкурентность и параллелизм

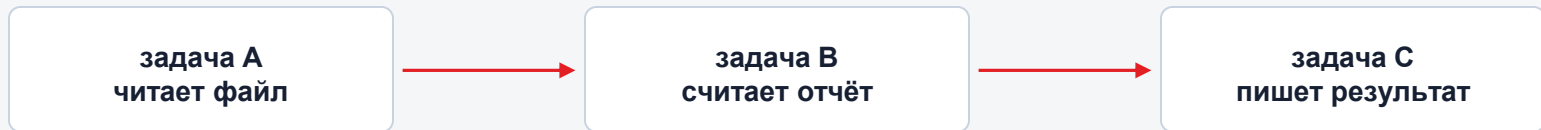
Это не одно и то же, но в реальных программах часто используются вместе

## Конкурентность

Программа умеет работать с несколькими задачами, которые логически выполняются одновременно: запросы, файлы, ожидание сети.

## Параллелизм

Несколько вычислений реально выполняются одновременно на разных ядрах процессора.



## В Scala

Основной стандартный инструмент для асинхронных задач — Future. Для запуска Future нужен ExecutionContext: он решает, где выполнять задачу.

# Future: асинхронное вычисление

Future[A] — значение типа A, которое появится позже

```
import scala.concurrent.{Future, Await}
import scala.concurrent.ExecutionContext.Implicits.global
import scala.concurrent.duration.*

def slowSquare(x: Int): Future[Int] = Future {
  Thread.sleep(500)
  x * x
}

val f = slowSquare(5)
println("Future started")

val result = Await.result(f, 2.seconds)
println(result) // 25
```

## Ключевые части

Future { ... } — запускает вычисление  
ExecutionContext — где выполнять  
Await.result — дождаться результата

## Для демонстрации

В учебных примерах Await удобен, чтобы увидеть результат в консоли. В настоящем сервисе блокирующее ожидание используют осторожно.

# Комбинирование Future

Асинхронные задачи можно связывать без ручного управления потоками

```
def loadName(id: Int): Future[String] =  
  Future { s"student-$id" }  
  
def loadScore(id: Int): Future[Int] =  
  Future { 70 + id }  
  
val oneStudent: Future[String] =  
  for  
    name <- loadName(1)  
    score <- loadScore(1)  
  yield s"$name: $score"  
  
val allScores: Future[List[Int]] =  
  Future.sequence(List(loadScore(1), loadScore(2), loadScore(3)))
```

## map / flatMap

Future поддерживает те же идеи, что и коллекции: преобразовать результат или связать следующий шаг.

## for-comprehension

Удобная форма для цепочки зависимых асинхронных вычислений.

## Future.sequence

Собирает много Future[A] в один Future[List[A]].

# Общая изменяемость и гонки

Параллельное выполнение опасно, если несколько задач меняют одно и то же состояние

```
var counter = 0

val tasks = (1 to 1000).map { _ =>
  Future {
    counter += 1 // небезопасно
  }
}

Await.result(Future.sequence(tasks), 5.seconds)
println(counter) // может быть меньше 1000
```

## Почему так

Операция `counter += 1` не атомарна: прочитать значение, увеличить, записать. Две задачи могут вмешаться друг в друга.

```
import java.util.concurrent.atomic.AtomicInteger

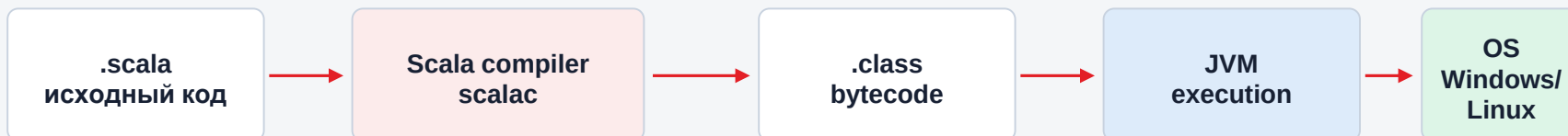
val safe = AtomicInteger(0)
Future { safe.incrementAndGet() }
```

## Scala-style

По возможности не делить mutable state. Лучше строить данные как immutable values и объединять результаты вычислений.

# JVM: что происходит под капотом

Scala работает в экосистеме JVM и компилируется в байткод



## Что это даёт

- можно использовать Java-библиотеки
- работает JIT-компиляция JVM
- управление памятью берёт на себя GC
- Scala-классы становятся JVM-классами

## Что важно помнить

Scala даёт более выразительный синтаксис, но ограничения JVM остаются: исключения, потоки, heap, stack, boxing и garbage collection.

# Память JVM: stack, heap, ссылки

В Scala нет ручного free/delete: памятью управляет JVM garbage collector

## Stack

Хранит кадры вызовов функций: параметры, локальные ссылки, адрес возврата. Глубокая рекурсия может переполнить стек.

## Heap

Основная область для объектов: List, case class, StringBuilder и другие экземпляры.

## References

Переменные часто хранят не сам объект, а ссылку на объект в heap.

```
val a = new StringBuilder("Scala")
val b = a
b.append(" JVM")
println(a) // Scala JVM
```

```
case class Point(x: Int, y: Int)
val p1 = Point(1, 2)
val p2 = p1.copy(x = 10)
```

## Garbage collector

Когда объект больше недостижим по ссылкам, JVM может освободить память автоматически. Точный момент освобождения мы не контролируем.

# Управление памятью в стиле Scala

Scala не управляет памятью вручную, но стиль кода сильно влияет на поведение программы

## Immutable-first

Меньше shared mutable state, проще reasoning, безопаснее при Future и потоках.

## tailrec

Хвостовая рекурсия помогает избежать роста стека и превращается компилятором в цикл.

## lazy val

Значение вычисляется только при первом обращении. Полезно для дорогих вычислений.

```
import scala.annotation.tailrec

@tailrec
def sum(n: Int, acc: Int = 0): Int =
  if n <= 0 then acc
  else sum(n - 1, acc + n)

lazy val expensive: String =
  println("computed once")
  "result"
```

## Практическая мысль

В Scala мы не освобождаем память вручную. Вместо этого пишем код так, чтобы объекты были короткоживущими, состояние не растекалось, а ресурсы закрывались.

# Итоговый пример: безопасный анализ файла

Пример объединяет ошибки, Using, assert, Future и запись результата

```
val readResult: Either[String, List[Student]] =
  readStudents("students.csv")

readResult match
  case Left(error) => println(s"Input error: $error")
  case Right(students) =>
    require(students.nonEmpty)
    val average = students.map(_._score).sum.toDouble / students.length
    assert(average >= 0)

val reports = Future.sequence(List(
  writeText("summary.txt", summary(students)),
  writeText("passed.txt", passed(students))
))

Await.result(reports, 3.seconds)
```

## Что внутри

- чтение файла через Using
- парсинг строк в Either
- проверка входных данных через require
- проверка результата через assert
- две записи в файл через Future.sequence

## Зачем такой пример

Он похож на реальную мини-задачу: входные данные могут быть плохими, отчёты можно писать параллельно, ресурсы нужно закрывать.

# Итоги третьей презентации

---

Теперь закрыты оставшиеся темы: ошибки, конкурентность, JVM и память

## Обработка ошибок

exceptions, Try, Either

## Параллелизм

асинхронные задачи и общая изменяемость

## assert

assert / require / ensuring

## JVM

байткод, Java interop, выполнение на JVM

## Конкурентность

Future, ExecutionContext, Future.sequence

## Память

stack, heap, ссылки, GC, immutable-first